

Capítulo 3: Modelado de Datos: Clases, Objetos y Traits

1. Explicación Teórica: El Molde de la POO Pura

Scala es un lenguaje que se adhiere al ideal de la **Programación Orientada a Objetos (POO) Pura: todo valor es un objeto**. Incluso los tipos que en Java serían primitivos (`Int`, `Double`) son objetos que descienden del tipo raíz `Any`.

En este contexto, las estructuras de Scala — `class`, `object`, y `trait` — sirven como los planos arquitectónicos para modelar el dominio de una aplicación, facilitando la encapsulación del estado y la lógica. La clave para un modelado idiomático en Scala radica en entender cómo estas estructuras se utilizan para **imponer la inmutabilidad**, **reemplazar los miembros estáticos** de Java y **favorecer la composición** sobre la herencia rígida.

2. Clases Estándar (`class`) y Constructores

Una `class` en Scala es el plano para crear instancias de objetos con su propio estado y métodos,.

El Constructor Primario

La principal diferencia con Java o Python es que **la declaración de la clase define el constructor primario**,.

- **Valores de Constructor Privados:** Si una variable de constructor se define sin `val` ni `var`, es un parámetro privado, no accesible fuera de la clase.
- **Campos Públicos Inmutables:** Si se antepone `val`, el parámetro se promociona automáticamente a un **campo público inmutable** (un *getter* de solo lectura).
- **Campos Públicos Mutables:** Si se antepone `var`, el parámetro se convierte en un **campo público mutable** (con *getter* y *setter*). **Esto se utiliza raramente** en código idiomático.

Sintaxis de la Clase Estándar

```
1 // El constructor primario toma dos argumentos: 'nombre' (val inmutable) y
  // 'antigüedad' (privado).
2 // Se asume que no queremos que 'antigüedad' sea accesible o modificable
  // desde fuera.
3 class Empleado(val nombre: String, antigüedad: Int) {
4
5     // Campo privado interno. Se utiliza 'var' porque el estado interno de
      un actor
```

```

6    // o clase simple a veces requiere mutabilidad controlada (aunque es
    mejor evitarlo).
7    private var salario: Double = 0.0
8
9    // Método público que accede a la propiedad pública 'nombre'.
10   def obtenerNombre: String = nombre
11
12   // Método auxiliar para el estado interno (opcional, para demostrar la
    mutabilidad).
13   def establecerSalario(monto: Double): Unit = {
14       salario = monto // La mutación de 'salario' ocurre internamente
15   }
16 }
17
18 val programador = new Empleado("Ana", 5) // Instanciación
19 println(programador.nombre)               // Acceso al campo 'nombre' (que
    es un val)
20 // programador.antiguedad                 // ERROR: 'antiguedad' es privado

```

3. Objetos Singleton (`object`) y Companion Objects

Scala elimina el concepto de miembros `static` que se encuentran en Java,. En su lugar, utiliza el patrón **Singleton** a nivel del lenguaje mediante la palabra clave `object` .

Un `object` garantiza que solo existe **una instancia** en la JVM. Esto es ideal para:

1. **Utilidades (Métodos Estáticos):** Funciones que no dependen del estado de ninguna instancia.
2. **Módulos (Namespaces):** Agrupar constantes y lógica.
3. **Entrada de Aplicación:** Un `object` que extiende `App` o contiene un método `main` es el punto de entrada de la aplicación.

Companion Objects

Cuando un `object` comparte el mismo nombre que una `class` y está definido en el mismo archivo, se convierte en su **Companion Object**. Esta relación es privilegiada: **el objeto y su clase pueden acceder a los miembros privados del otro**.

- **Patrón Factory (`apply`):** El uso más común del Companion Object es definir un método `apply` . Si el objeto tiene un método `apply` , se puede invocar como si fuera el constructor de la clase, **sin usar la palabra clave `new`** .

Sintaxis del Companion Object y `apply`

```

1    // Clase. Nótese que el constructor es privado.

```

```

2  class ConexiónDB private (val url: String, val timeout: Int)
3
4  // Companion Object: Accede al constructor privado y ofrece una factoría
   segura.
5  object ConexiónDB {
6
7      // 1. Método apply para instanciar sin 'new'
8      def apply(url: String): ConexiónDB =
9          new ConexiónDB(url, 5000) // Llama al constructor privado con timeout
   por defecto
10
11     // 2. Miembro que se comportaría como un "static final"
12     val CONEXION_LOCAL = ConexiónDB("jdbc:local")
13 }
14
15 // Uso: Se llama a ConexiónDB.apply("url")
16 val prodDb = ConexiónDB("jdbc:produccion:9000")

```

4. La Potencia de los Case Classes (`case class`)

Los **Case Classes** son una de las características más distintivas de Scala y la forma idiomática de modelar **datos inmutables**,. Fueron diseñados específicamente para ser utilizados en conjunción con el *Pattern Matching*.

La adición de la palabra clave `case` delante de una clase instruye al compilador a generar automáticamente código repetitivo (*boilerplate*) que Java requeriría manualmente.

Características Automáticas de `case class` :

1. **Constructor Simplificado:** El método `apply` se genera en el Companion Object, permitiendo la instanciación sin `new`.
2. **Inmutabilidad:** Todos los parámetros del constructor primario se convierten implícitamente en campos `val` públicos.
3. **Igualdad Estructural (`equals` y `hashCode`):** Compara el contenido de los campos, no solo la referencia.
4. **Representación en Cadena (`toString`):** Genera una representación legible que incluye los nombres de los campos.
5. **Copia Modificada (`copy`):** Crea un nuevo objeto inmutable a partir del original, permitiendo la modificación de campos específicos.
6. **Desestructuración (`unapply`):** Genera un método para la **desestructuración** automática en el *Pattern Matching*.

Sintaxis de Case Class

```

1 // Declaración concisa para modelar un registro de datos inmutable.
2 case class Transacción(id: Long, monto: BigDecimal, moneda: String,
   completada: Boolean = false)
3
4 // 1. Uso sin 'new' (gracias a apply en el Companion Object generado)
5 val tx1 = Transacción(100, BigDecimal(500.0), "USD")
6
7 // 2. Creación de una copia modificada (el original tx1 es inmutable)
8 // Se usa tx1 como la base, solo se cambia 'completada'.
9 val tx2 = tx1.copy(completada = true)
10 // tx2 es un nuevo objeto: Transacción(100, 500.0, "USD", true)

```

Comparativa con Clases de Datos en Java/Python

Característica	Java (POO/JDK 17+)	Python (Dinámico/dataclass)	Scala (Idiomático/Case Class)
Declaración	<code>public record T(...) {}</code>	<code>@dataclass class T:</code>	<code>case class T(...) .</code>
Inmutabilidad	Garantizada (por <code>record</code> / <code>final</code>).	Por convención o librerías externas.	Garantizada por defecto.
Método <code>copy</code>	No generado automáticamente.	No generado automáticamente.	Generado automáticamente.
Uso en Pattern Matching	Parcial (disponible en <code>switch</code> de Java).	No aplica directamente (lenguaje dinámico).	Integración nativa y potente (gracias a <code>unapply</code>).

5. Traits (`trait`): Composición y Abstracción

Los **Traits** son el mecanismo de Scala para la **herencia de comportamiento** y actúan como interfaces enriquecidas,.

- **Flexibilidad:** Un `trait` puede contener **métodos abstractos** (sin implementación) y **métodos concretos** (con implementación).
- **Mixins:** Una clase solo puede heredar de una única clase base (herencia simple), pero puede **mezclar** (`mix in`) **múltiples traits** usando la palabra clave `with` .
- **Composición:** Este mecanismo de *mixins* resuelve el problema de la herencia múltiple de Java, favoreciendo la **composición de comportamientos** modulares.

Traits con Argumentos (Scala 3)

Una de las adiciones significativas en Scala 3 es que los `traits` ahora pueden aceptar **parámetros de constructor**. Esto permite que el *trait* capture información al ser mezclado en una clase, haciendo que la composición sea más potente.

Sintaxis de Traits y Uso

```
1 // 1. Definición de un trait (comportamiento)
2 trait Validador[T] { // T es un tipo genérico que se validará
3     def validar(data: T): Boolean
4     // Método concreto (con implementación por defecto)
5     def esRequerido: Boolean = true
6 }
7
8 // 2. Trait con argumento (Scala 3)
9 trait Trazabilidad(val idOperacion: String):
10     def log(mensaje: String): Unit = println(s"$idOperacion $mensaje")
11
12 // 3. Clase que hereda de una clase base y mezcla dos traits (mixins)
13 class Servicio(nombre: String)
14     extends ServicioBase(nombre) // Hereda de una clase base (máximo una)
15     with Validador[String]       // Mezcla el primer trait
16     with Trazabilidad("TX-456")  // Mezcla el segundo trait (con argumento
    de constructor)
17 {
18     override def validar(data: String): Boolean = data.nonEmpty //
    Implementa el método abstracto
19
20     def procesar(data: String): Unit =
21         if (validar(data)) log(s"Datos válidos: $data")
22 }
```

6. Best Practices (*The Scala Way*)

El modelado de datos en Scala es un acto de equilibrio entre la concisión y la expresividad, dictado por el paradigma funcional-orientado a objetos.

Objetivo de Modelado	Herramienta Idiomática	Justificación
Modelos de Datos Inmutables	case class	Genera automáticamente <i>boilerplate</i> (<code>equals</code> , <code>copy</code>) y habilita la desestructuración y la inmutabilidad por defecto,.
Comportamiento Reutilizable	trait y with	Permite la composición modular de funcionalidades sin la rigidez de la

Objetivo de Modelado	Herramienta Idiomática	Justificación
		herencia de clases,.
Miembros Estáticos y Factories	<code>object</code> (Singleton/Companion)	Reemplaza a <code>static</code> y debe contener los métodos de fábrica (<code>apply</code>) para la clase, asegurando una única instancia de la utilidad.
Inmutabilidad de Campos	<code>val</code> en constructores	Declarar <code>val</code> por defecto en el constructor primario; evita <code>var</code> a menos que sea un estado interno de baja visibilidad (como en un Actor).

El enfoque de Scala es usar `case class` para *qué es* el dato (modelado) y `trait` para *qué puede hacer* el dato (comportamiento). Al preferir los `trait` s para la herencia y la composición, el desarrollador se enfoca en ensamblar capacidades, como si estuviera construyendo con bloques de Lego.